📚 Library Management API

Mini Project — Self Testing

JavaScript → Express → MongoDB → JWT → Role-Based Auth

Project Overview

Build a complete **Library Management REST API** that manages users, books, and borrowing records. This project tests everything you learned — from JavaScript basics to JWT authentication.

Concept Tested	Where it appears
async / await / try / catch	Every controller function
Array methods	Filtering, mapping results
Express routes & middleware	All route files
MongoDB — all 6 methods	All controllers
Query operators ($gt, $in...)	Book filtering, borrow logic
JWT Authentication	All protected routes
Role-based access	Admin vs Librarian
Model relationships	Borrow connects User + Book
timestamps	borrowedAt, returnedAt, createdAt
Real logic	availableCopies, status checks

Collections — 3 Models

1 — User Model

Field	Type	Rules
name	String	required

Field	Type	Rules
email	String	required, unique
password	String	required — must be hashed
role	String	enum: ['admin', 'librarian'] — default: 'librarian'

```
{ timestamps: true, versionKey: false }
```

2 — Book Model

Field	Type	Rules
title	String	required
author	String	required
genre	String	required (ex: 'Science', 'History', 'Fiction')
totalCopies	Number	required — total copies in library
availableCopies	Number	required — copies available to borrow

```
{ timestamps: true, versionKey: false }
```

💡 availableCopies starts equal to totalCopies. Decreases when borrowed, increases when returned.

3 — Borrow Model

Field	Type	Rules
user	ObjectId	ref: 'User' — who borrowed
book	ObjectId	ref: 'Book' — which book
borrowedAt	Date	default: Date.now — when borrowed
returnedAt	Date	default: null — filled when returned
status	String	enum: ['borrowed', 'returned'] — default: 'borrowed'

```
{ timestamps: true, versionKey: false }
```

All Routes

Auth Routes — public (no token needed)

Method	Route	What it does
POST	/auth/register	Register new user (librarian or admin)
POST	/auth/login	Login and get JWT token

User Routes — protected

Method	Route	Who can access	What it does
GET	/users	admin only	Get all users
GET	/users/:id	admin + owner	Get one user
DELETE	/users/:id	admin only	Delete a user

Book Routes — protected

Method	Route	Who can access	What it does
GET	/books	all users	Get all books
GET	/books/:id	all users	Get one book
GET	/books?genre=...	all users	Filter books by genre
GET	/books?author=...	all users	Filter books by author
POST	/books	admin only	Add new book
PUT	/books/:id	admin only	Update book info
DELETE	/books/:id	admin only	Delete a book

Borrow Routes — protected

Method	Route	Who can access	What it does
POST	/borrows	all users	Borrow a book
PUT	/borrows/:id/return	all users	Return a borrowed book

Method	Route	Who can access	What it does
GET	/borrows	admin only	Get all borrow records
GET	/borrows/my	all users	Get MY borrow records

The Real Logic — Most Important Part

Borrow a Book — POST /borrows

This is the hardest part. Follow these steps in order:

```
Step 1 → Get bookId from req.body
Step 2 → Find the book by id — if not found → 404
Step 3 → Check book.availableCopies > 0
         if 0 → return 400 'No copies available'
Step 4 → Create borrow record:
         { user: req.user.id, book: bookId, status: 'borrowed' }
Step 5 → Decrease availableCopies by 1:
         { $inc: { availableCopies: -1 } }
Step 6 → Return the borrow record
```

> ⚠ Step 4 and Step 5 must BOTH succeed. If one fails — the data becomes inconsistent.

Return a Book — PUT /borrows/:id/return

Follow these steps in order:

```
Step 1 → Find borrow record by id — if not found → 404
Step 2 → Check borrow.status === 'borrowed'
         if 'returned' already → 400 'Book already returned'
Step 3 → Check borrow.user equals req.user.id
         if different → 403 'This is not your borrow record'
Step 4 → Update borrow record:
         { status: 'returned', returnedAt: Date.now() }
Step 5 → Increase availableCopies by 1:
         { $inc: { availableCopies: 1 } }
Step 6 → Return updated borrow record
```

Filter Books — GET /books?genre=...

Use **req.query** to get filter from URL. Build the filter object dynamically:

```
const getAllBooks = async (req, res) => {
    const filter = {};

```

```
    if (req.query.genre)  filter.genre  = req.query.genre;
    if (req.query.author) filter.author = req.query.author;

    const books = await Book.find(filter);
    res.status(200).json(books);
};
```

Middleware — Two Files

authMiddleware.js — you already know this

```
// checks JWT token
// adds req.user = { id, email, role }
// calls next() if valid
```

> Add role to the token payload: jwt.sign({ id, email, role }, SECRET)

roleMiddleware.js — NEW

A middleware that checks if the user has the required role.

```
const allowOnly = (...roles) => {
    return (req, res, next) => {
        if (!roles.includes(req.user.role)) {
            return res.status(403).json({
                message: 'Access denied — not authorized'
            });
        }
        next();
    };
};

module.exports = allowOnly;
```

How to use it in routes:

```
const protect   = require('../middleware/authMiddleware');
const allowOnly = require('../middleware/roleMiddleware');

// admin only
router.post('/books', protect, allowOnly('admin'), createBook);

// admin and librarian
router.get('/books',  protect, allowOnly('admin', 'librarian'), getAllBooks);
```

Project Structure

```
library-api/
├── src/
│   ├── controllers/
│   │   ├── authController.js     ← register, login
│   │   ├── userController.js     ← getAll, getOne, delete
│   │   ├── bookController.js     ← CRUD + filter
│   │   └── borrowController.js   ← borrow, return, getAll, getMy
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── userRoutes.js
│   │   ├── bookRoutes.js
│   │   └── borrowRoutes.js
│   ├── models/
│   │   ├── userModel.js
│   │   ├── bookModel.js
│   │   └── borrowModel.js
│   ├── middleware/
│   │   ├── authMiddleware.js     ← protect routes
│   │   └── roleMiddleware.js     ← admin / librarian check
│   └── db/
│       └── database.js
├── app.js
├── server.js
└── package.json
```

Setup

```
mkdir library-api
cd library-api
npm init -y
npm install express mongoose jsonwebtoken bcryptjs
```

Testing Order — with curl

Follow this order — each step depends on the previous:

Step 1 — Register admin

```
curl -X POST http://localhost:3000/auth/register \
     -H 'Content-Type: application/json' \
     -d
'{"name":"Admin","email":"admin@lib.com","password":"123456","role":"admin"}'
```

Step 2 — Register librarian

```
curl -X POST http://localhost:3000/auth/register \
```

```
     -H 'Content-Type: application/json' \
     -d
'{"name":"Ali","email":"ali@lib.com","password":"123456","role":"librarian"}'
```

Step 3 — Login as admin → save token

```
curl -X POST http://localhost:3000/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"email":"admin@lib.com","password":"123456"}'
```

Step 4 — Add books (admin token required)

```
curl -X POST http://localhost:3000/books \
     -H 'Content-Type: application/json' \
     -H 'Authorization: Bearer ADMIN_TOKEN' \
     -d '{"title":"Clean Code","author":"Robert
Martin","genre":"Programming","totalCopies":3,"availableCopies":3}'
```

Step 5 — Login as librarian → save token

```
curl -X POST http://localhost:3000/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"email":"ali@lib.com","password":"123456"}'
```

Step 6 — Borrow a book (librarian token)

```
curl -X POST http://localhost:3000/borrows \
     -H 'Content-Type: application/json' \
     -H 'Authorization: Bearer LIBRARIAN_TOKEN' \
     -d '{"bookId":"BOOK_ID_HERE"}'
```

Step 7 — Return the book

```
curl -X PUT http://localhost:3000/borrows/BORROW_ID/return \
     -H 'Authorization: Bearer LIBRARIAN_TOKEN'
```

Step 8 — Filter books by genre

```
curl http://localhost:3000/books?genre=Programming \
     -H 'Authorization: Bearer LIBRARIAN_TOKEN'
```

Step 9 — Librarian tries admin route → should fail

```
curl -X GET http://localhost:3000/users \
     -H 'Authorization: Bearer LIBRARIAN_TOKEN'


Expected: 403 Access denied ✕
```

Difficulty Map

Part	Difficulty	Why
Models (3 files)	☆☆	Similar to what you did before
Auth routes	☆☆	Same as before
Book CRUD	☆☆	Standard CRUD
Book filtering (query)	☆☆☆	Dynamic filter object
roleMiddleware	☆☆☆	New concept — roles
Borrow a book	☆☆☆☆	Logic + update two collections
Return a book	☆☆☆☆	Checks + update two collections
Get MY borrows	☆☆☆	Filter by req.user.id

Hints — Read Only If Stuck

Hint 1 — Borrow logic

> 💡 Use $inc: { availableCopies: -1 } to decrease. Use $inc: { availableCopies: 1 } to increase.

Hint 2 — Get MY borrows

```
const borrows = await Borrow.find({ user: req.user.id });
```

Hint 3 — Check borrow belongs to user

```
if (borrow.user.toString() !== req.user.id) {
    return res.status(403).json({ message: 'Not your record' });
}
```

> ⚠ Use .toString() because borrow.user is an ObjectId — not a plain string.

Hint 4 — Add role to token

```
jwt.sign({ id: user._id, email: user.email, role: user.role }, SECRET, { expiresIn:
'7d' });
```

Hint 5 — roleMiddleware pattern

```
const allowOnly = (...roles) => (req, res, next) => {
    if (!roles.includes(req.user.role))
        return res.status(403).json({ message: 'Access denied' });
    next();
};
```

Self-Test Checklist

Task	Done?
3 models created with correct fields	☐
timestamps: true and versionKey: false on all	☐
register and login working	☐
role added to JWT token	☐
authMiddleware working	☐
roleMiddleware working	☐
Book CRUD working	☐
Book filter by genre working	☐
Book filter by author working	☐
Borrow book → availableCopies decreases	☐
Cannot borrow if availableCopies = 0	☐
Return book → availableCopies increases	☐
Cannot return already returned book	☐
Cannot return someone else's borrow	☐

Task	Done?
GET /borrows/my returns only MY records	☐
Librarian cannot access admin routes	☐

Build it yourself — you have everything you need! 💪